

ShipNet — Guía de trabajo: Baruc Herbas

Rol: Backend · Sprint 1 (11/09 - 15/09/2026) · Programación Web, UPDS Cochabamba

1. Cómo subir tus cambios al repositorio

El proyecto vive en github.com/angelops1998/shipnet. Nadie sube cambios directo a la rama **main**: cada tarea (issue) se trabaja en su propia rama y se integra mediante un **Pull Request** (PR) que otro integrante revisa antes de aprobar.

1.1 Configuración inicial (solo la primera vez)

1. Acepta la invitación como colaborador que te llegó por correo (o entra a github.com/angelops1998/shipnet/invitations).

2. Clona el repositorio en tu computadora:

```
git clone https://github.com/angelops1998/shipnet.git
cd shipnet
```

3. Crea tu archivo de configuración local a partir de la plantilla. Este archivo **no se sube** a GitHub (está en el `.gitignore`), así que cada uno pone su propia contraseña de MySQL:

```
cp appsettings.Development.json.example appsettings.Development.json
```

Luego abre `appsettings.Development.json` y reemplaza **TU_PASSWORD_AQUI** por la contraseña de tu MySQL local.

4. Verifica que el proyecto compile:

```
dotnet build
```

1.2 Flujo para cada issue

Repite estos pasos por cada issue que tomes. Usa el número de issue (SN-XX) en el nombre de la rama para que sea fácil identificarla.

1. Ponte en **main** y trae lo último que subió el equipo:

```
git checkout main
git pull
```

2. Crea una rama nueva para tu issue (ejemplo con SN-01):

```
git checkout -b feature/sn-01-modelo-er
```

3. Trabaja normalmente. Cuando tengas un avance que valga la pena guardar, revisa qué cambió y haz un commit con un mensaje claro:

```
git status
git add .
git commit -m "SN-01: diagrama ER con entidades y relaciones"
```

Antes de **git add .** mira la lista de **git status**: si aparece `appsettings.Development.json`, `bin/` u `obj/`, algo está mal con el `.gitignore` — avisa antes de commitear.

4. Sube tu rama a GitHub (la primera vez con **-u**; las siguientes basta **git push**):

```
git push -u origin feature/sn-01-modelo-er
```

1.3 Crear el Pull Request

1. Entra a **github.com/angelops1998/shipnet**. Verás un aviso amarillo con tu rama y el botón **Compare & pull request**. Haz clic.
2. **Título:** el mismo del issue, por ejemplo *SN-01: Diseñar el modelo entidad-relación*.
3. **Descripción:** resume qué hiciste y agrega la línea **Closes #N** (donde N es el número del issue en GitHub). Así el issue se cierra solo cuando el PR se aprueba.
4. En **Reviewers** elige a un compañero para que revise. Haz clic en **Create pull request**.
5. Si el revisor pide cambios, corrígelos en la misma rama, haz commit y push de nuevo — el PR se actualiza solo.
6. Cuando esté aprobado, se hace clic en **Merge pull request** y luego **Delete branch**.

1.4 Reglas del equipo

- Nunca hacer **git push** directo a **main**.
- Nunca subir **appsettings.Development.json** (contiene tu contraseña de MySQL).
- Un issue = una rama = un Pull Request.
- Hacer **git pull** en main antes de crear cada rama nueva, para no trabajar sobre código viejo.
- Si al hacer pull aparece un **conflicto**, no lo resuelvas a ciegas: avisa en el grupo.

2. Tu tarea del Sprint 1

Tienes un solo issue este sprint, pero es la base de toda la seguridad del sistema: sin Identity no hay login ni roles, y sin roles no se puede distinguir qué puede hacer un docente frente a un estudiante. Total: **3 puntos**.

SN-06 — Configurar ASP.NET Core Identity con roles

Sprint	Sprint 1 (11/09 - 15/09)
Puntos	3
Rama sugerida	feature/sn-06-...

¿Qué es Identity?

Es el sistema de autenticación que trae ASP.NET Core: registro, login, contraseñas hasheadas y **roles**. Para ShipNet necesitamos tres roles: **Admin**, **Docente** y **Estudiante**. Con eso, más adelante bastará escribir **[Authorize(Roles = "Docente")]** en un controlador para que solo un docente pueda, por ejemplo, abrir una sesión de asistencia.

Pasos tentativos

1. Instala los paquetes (si Angel ya los agregó en SN-00, este paso no hace nada):

```
dotnet add package Microsoft.AspNetCore.Identity.EntityFrameworkCore --version 8.0.10
dotnet add package Pomelo.EntityFrameworkCore.MySql --version 8.0.2
```

2. Crea **Models/ApplicationUser.cs**. Extiende el usuario base de Identity para poder agregar campos propios:

```
using Microsoft.AspNetCore.Identity;

namespace shipnet.Models;

public class ApplicationUser : IdentityUser
{
    public string NombreCompleto { get; set; } = string.Empty;
}
```

3. Asegúrate de que **Data/ApplicationDbContext.cs** (lo crea Daniel en SN-03) herede de **IdentityDbContext<ApplicationUser>**. Si Daniel aún no lo tiene, créalo tú con el constructor básico y él agrega sus DbSet después — coordínenlo para no pisarse.

4. Registra Identity en **Program.cs**, después del AddDbContext y antes de **builder.Build()**:

```
using Microsoft.AspNetCore.Identity;
using shipnet.Data;
using shipnet.Models;

builder.Services.AddIdentity<ApplicationUser, IdentityRole>(options =>
{
    options.Password.RequireNonAlphanumeric = false;
    options.Password.RequiredLength = 6;
})
.AddEntityFrameworkStores<ApplicationDbContext>()
.AddDefaultTokenProviders();

builder.Services.ConfigureApplicationCookie(options =>
{
    options.LoginPath = "/Account/Login";
    options.AccessDeniedPath = "/Account/AccessDenied";
});
```

```
});
```

5. En el pipeline (después de **app.UseRouting()**) agrega **UseAuthentication**. El orden importa: autenticación va **antes** de autorización:

```
app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();
```

6. Crea las tablas de Identity en MySQL. Como el equipo no usa migraciones, genera el SQL una sola vez para pasárselo a Daniel (él lo integra en **Database/schema.sql**):

```
dotnet tool install --global dotnet-ef --version 8.0.10
dotnet ef migrations add TempIdentity
dotnet ef migrations script -o identity.sql
dotnet ef migrations remove
```

Ese comando genera el CREATE TABLE de AspNetUsers, AspNetRoles, AspNetUserRoles, AspNetUserClaims, etc. Envíale **identity.sql** a Daniel y **no subas la carpeta Migrations/** al repo (el último comando la elimina).

7. Siembra los roles al arrancar la app. Crea **Data/SeedData.cs**:

```
using Microsoft.AspNetCore.Identity;

namespace shipnet.Data;

public static class SeedData
{
    public static async Task CrearRolesAsync(IServiceProvider services)
    {
        var roleManager = services.GetRequiredService<RoleManager<IdentityRole>>();
        foreach (var rol in new[] { "Admin", "Docente", "Estudiante" })
        {
            if (!await roleManager.RoleExistsAsync(rol))
                await roleManager.CreateAsync(new IdentityRole(rol));
        }
    }
}
```

Y en **Program.cs**, justo después de **var app = builder.Build();**:

```
using (var scope = app.Services.CreateScope())
{
    await SeedData.CrearRolesAsync(scope.ServiceProvider);
}
```

8. Crea **Controllers/AccountController.cs** con las acciones **Login** (GET y POST), **Register** (GET y POST) y **Logout** (POST). Usa **SignInManager<ApplicationUser>** para login/logout y **userManager<ApplicationUser>** para crear usuarios y asignar rol con **AddToRoleAsync(user, "Estudiante")**.
9. Crea ViewModels simples (**LoginViewModel**, **RegisterViewModel**) con Email, Password y DataAnnotations ([Required], [EmailAddress]). Las vistas Razor las diseña Bianca en Sprint 2 (SN-14); para probar este sprint puedes hacer vistas mínimas sin estilo.
10. Prueba: registra un usuario, inicia sesión, cierra sesión. Verifica en MySQL que aparezca en **AspNetUsers** y su rol en **AspNetUserRoles**.

Criterio de aceptación

- [] Login, registro y logout funcionando
- [] Roles Admin, Docente y Estudiante creados automáticamente al arrancar

- [] identity.sql entregado a Daniel para el schema.sql
- [] Un controlador de prueba con [Authorize(Roles = "Admin")] bloquea a quien no es Admin